



KUBERNETES NOTES

MY LEARNING NOTES FOR

• DEVOPS & CLOUD INTERVIEWS •

TOPICS COVERED

- | | |
|---|--|
|  Deployments |  ConfigMaps |
|  ReplicaSets |  Secrets |
|  DaemonSets |  Probes |
|  Jobs & CronJobs |  Taints & Tolerations |
|  Persistent Volumes (PV) |  HPA & VPA |
|  Services |  RBAC |
|  Ingress |  Helm |
|  StatefulSets | |

CREATED & HANDWRITTEN BY

Anuja Kanase

Learn • Practice • Revise • Succeed

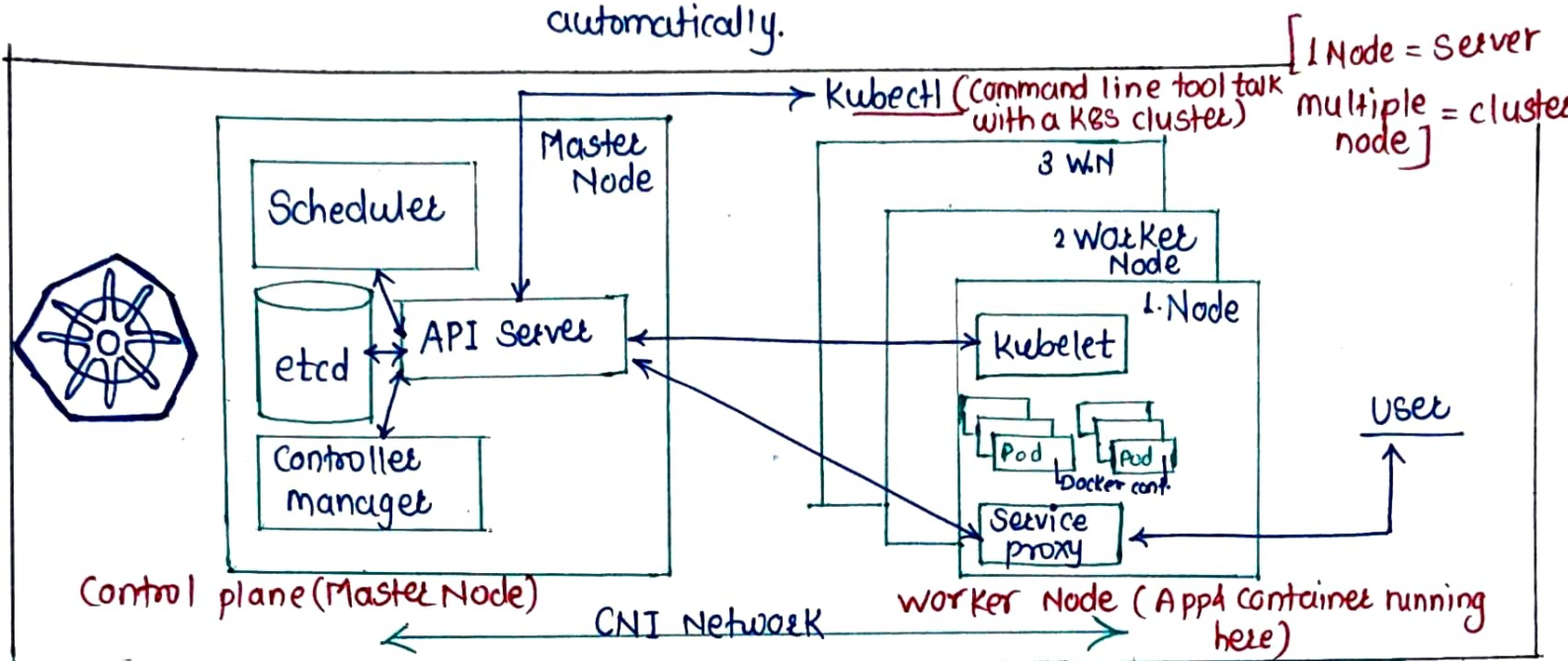


Learning Kubernetes one concept at a time.



Kubernetes Architecture

Kubernetes architecture is a system where control plane (master node) manages worker nodes to run and maintain containerized applications automatically.



1 The Control Plane (Master Node)

- **API Server** : The endpoint of Kubernetes, whenever we want to interact with cluster, request goes through the API Server. It validates & processes these req. to backend components.
- **etcd** : Database of K8S, It stores cluster info. i.e. what pods are running, their status.
- **Scheduler** : It decides which worker node should run the container when we ask container to be run. It considers resource requirement and availability & hardware/software limits.
- **Controller Manager** : Auto-healing manager. If something breaks it fixes automatically. It checks continuously if app is running, did any container crash? Is desired no. of pods available.

2. Worker Node (where actual applications run)

- **Kubelet** : An agent that runs on each node in the cluster. It ensures all containers running inside the pods are healthy.
- **Kube-Proxy** : Handles networking between applications. Allows network communication to pods from inside or outside of cluster. It handles traffic routing & load balancing.
- **Pod** : group of one or more containers.
- **Container Runtime** : Software that runs containers. (eg. Docker, containerd, CRI-O)

CNI : Communicator between nodes (i.e. Master node & worker node)
(Container Network Interface)

Namespace :-

A Namespace in K8S is used to divide cluster into multiple virtual env. It helps to organize resources, separate team/projects, managing resource better ~~separate~~ avoid naming conflicts.

eg. one company has: Dev Team, Testing Team, Production Team

Instead of creating 3 clusters, they create 3 namespace :-

i.e. dev
test
prod

each team works independently

- `kubectl get namespaces` [To view namespaces]
By default there are 4 ns available in K8S cluster
i.e. 1) default
2) kube-node-lease
3) kube-public
4) kube-system
- `kubectl create namespace dev` [To create namespace]
- `kubectl delete namespace dev` [To delete namespace]
- `kubectl run nginx --image=nginx -n dev` [To run pod in namespace]
- `kubectl get pods -n dev` [To view pods in namespace]

YAML OR Manifest File

Vim
namespace
-yml

```
apiVersion: v1
kind: Namespace
metadata:
  name: nginx
```

- `kubectl apply -f namespace.yaml` [Apply]
- `kubectl create ns nginx` [command line]
- `kubectl config set-context --current --namespace=dev`
[To set default namespace temporarily]

Pod

A pod is the smallest deployable unit in K8s.

- ✓ It is where your application actually runs.
- ✓ It contains: one or more containers, storage, Network IP

eg. Suppose you run: nginx website, node.js app, Redis
each app. can run inside separate pods.

Pod 1 → Nginx Pod 2 → Node.js Pod 3 → Redis

- ✓ It helps to run app, scale apps, restart failed containers, manage networking handle deployment easily
- ✓ Containers inside same pod share same IP, same port space, storage Vol^m
- ✓ Pods can share data between containers.
- ✓ K8s continuously checks pod health

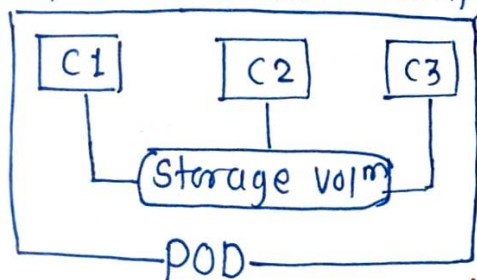
Pending → Running → Succeeded/Failed [Pod lifecycle]

- Vim pod.yml

```
Kind: Pod
apiVersion: v1
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

- Multi-container pod

one pod can contain multiple



containers.

eg. Suppose your website crashes

K8s automatically: Detects failed Pod
creates new Pod
keep app running.

may get new IP that's why we use **Deployments & Services** instead of directly managing pods.

[manage pods]

[Expose pods to users]

- To create Pod

• `kubectl run nginx --image=nginx`

- To view Pod

• `kubectl get pods`

- To view detailed info of Pod

• `kubectl describe pod/nginx-pod -n nginx`

- To view logs

• `kubectl logs nginx`

- To delete pod

• `kubectl delete pod nginx`

- `kubectl apply -f pod.yml`

- To go inside pod

• `kubectl exec -it pod/nginx-pod -n nginx -- bash`

Deployment

- A Deployment is a K8s resource that manages the lifecycle of pods. It ensures the desired no. of pods are running and provides features like scaling, rolling updates and rollbacks.
- It manages automatic pod management, self-healing, easy scaling, zero downtime updates, rollback supports.

YAML

```
apiVersion: apps/v1
Kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3 (no. of pod copies)
  selector: (Identifies Pods managed by deployment)
    matchLabels:
      app: nginx
  template: (Pod definition)
    metadata:
      labels: (used for pod matching)
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

1. Scaling - we can easily scale app (scale to 5 pods)
Kubectl scale deployment nginx-deployment --replica=5
 2. Rolling update - update app without downtime
(update image)
Kubectl set image deployment/nginx-deployment
nginx=nginx:1.26
(K8s gradually replace old pods with new ones)
 3. Rollbacks - if something breaks, revert quickly
Kubectl rollout undo deployment/nginx-deployment
 4. Self-Healing - if pod crashes, K8s automatically replaces it to maintain desired state.
- To create deployment
Kubectl apply -f deployment.yml
 - To view deployment
Kubectl apply -f get deployments
 - To describe deployment
Kubectl describe deployment nginx-deployment
 - To delete deployment
Kubectl delete deployment nginx-deployment

- When to use deployments

Use it for stateless application (APIs, webapps), microservices, scalable workloads

- Avoid deployment for stateful apps (mysql, mongodb)

Real-time Scenario: A Node.js app is running with 3 pods, management asks you to upgrade app version, ensure no downtime.

1) Kubectl set image deployment/nodejs-app app=nodejs:v2

monitor:- 2) Kubectl rollout status deployment/nodejs-app

if issues occur: 3) Kubectl rollout undo deployment/nodejs-app

ReplicaSet

- It ensures a specified no. of identical pods are running at all times.
- It maintain desired no. of pod replicas and replaces pods if they fail or are deleted.

Why we need Replicasets?

Imagine you have 3 pods running, if pod-2 crashes **The ReplicaSet** detects that only
(Pod-1, Pod-2, Pod-3) (Pod-1, Pod-3)

2 pods are running instead of 3 and automatically creates a new pod.

(Pod-1, Pod-2 (new), Pod-3)

This ensures high availability

- It continuously checks, Are 3 pods running?
if Pods < 3 → create Pod
if Pods > 3 → delete Pod
- ReplicaSet identifies pods using labels.

✓ Real-Time Scenario

Your app requires 4 running pods. one worker node crashes and 2 pods are lost what happens?

- 1) ReplicaSet detects only 2 pods are available
- 2) Desired count is 4
- 3) ReplicaSet creates 2 new pods on healthy nodes
- 4) App. remains available.

DaemonSet

- It ensures one Pod runs on every node in the cluster.
- It deploys and maintains one copy of a pod on each worker node

How IT Works?

Suppose you have 3 worker nodes: A Daemonset creates

Worker node 1 ---> Pod 1

Worker node 2 ---> Pod

Worker node 3 ---> Pod

if a new node joins Worker node 4 ---> Pod (Automatically created)

if a node is removed Associated pod is removed automatically

~~YAML~~ ~~No replicas~~ because Daemonset automatically creates one pod per node.

Job

- It runs a task once and ensures it completes successfully.
- It creates one or more pods and keeps retrying until the task is completed successfully.

Why do we need Jobs:-

Deployments are used for applications that should run continuously.

Some tasks need to run only once:

- ✓ Database backup
- ✓ Report generation
- ✓ Sending emails.
- ✓ Data migration
- ✓ Batch processing

for such tasks, we use a Job.

YAML

```
apiVersion: batch/v1
Kind: Job
metadata:
  name: backup-job
Spec:
  template:
    spec:
      containers:
        - name: backup
          image: busybox
          command: ["echo", "Database Backup Completed"]
      restartPolicy: Never (Pod restart behavior) ONFailure
```

(Task to execute)

Parallel Jobs.

Sometimes we want multiple tasks to run simultaneously.

eg. YAML

```
Spec:
  completions: 5
  parallelism: 2
```

meaning:- Total task - 5
Run at a time - 2

K8s completes all 5 tasks by running 2 in parallel

Real-Use Case:

- ✓ Database Backup Every night: create backup, store backup, exit
- ✓ Database Migration move data from old DB to new DB, exit
- ✓ Report Generation Generate report, send report, exit
- ★ To create Job `kubectl apply -f job.yml`
- To view Job `kubectl get jobs`
- To describe Job `kubectl describe job backup-job`
- To delete Job `kubectl delete job backup-job`

Cronjob

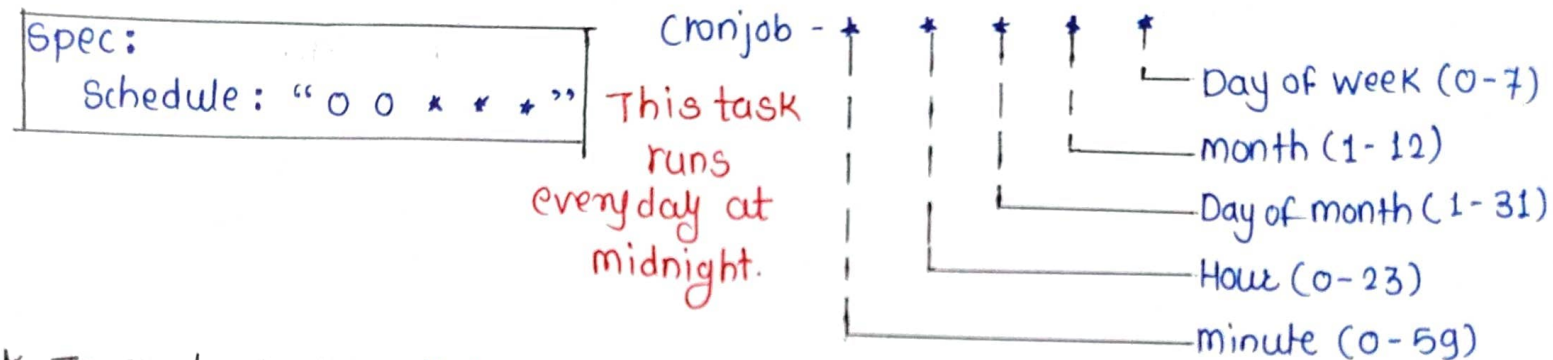
- Used to run jobs on scheduled basis
- It automatically creates and runs job at specified time or intervals.
- Why we need Cronjobs.

Many task need to run repeatedly at fixed times.

- ✓ Daily database backups
- ✓ cleaning temp files
- ✓ scheduled data synchronization
- ✓ Sending reports every morning
- ✓ Log rotation

Instead of manually running jobs, cronjob automate the process.

YAML



- * To create Cronjob `kubectl apply -f cronjob.yaml`
- To view Cronjob `kubectl get cronjobs`
- To Describe Cronjob `kubectl describe cronjob backup-cronjob`
- To delete Cronjob `kubectl delete cronjob backup-cronjob`

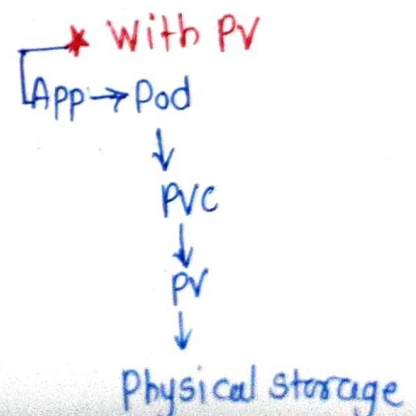
Persistent Volume (PV)

- It is storage resource in K8S that exists independently of pods.
- It is piece of storage in K8S cluster that stores data permanently even if pods are deleted or restarted.

Why we need PV?

By default, Pod storage is ephemeral. So if pod is deleted all the data inside pod (i.e. Database, User uploads, Log files, app. data) will be lost, so PV solve this problem.

- * To create PVs `kubectl apply -f persistentvolume.yaml`
- To view PVs `kubectl get pv`
- To Describe PVs `kubectl describe pv my-pv`
- To Delete PV `kubectl delete pv my-pv`



Persistent Volume Claim (PVC)

• It is request for storage. PVC is how a pod asks for storage from PV.

PV → Storage Provider

PVC → Storage Request

Pod → Storage User

• Pods cannot directly manage provision storage. PVCs act as bridge between Pod and PV. PVC's can specify storage size requirements, such as 5Gi or 10Gi, Access modes

including :- ReadWriteOnce (RWO) - mounted as read-write by single node
ReadOnlyMany (ROX) - mounted as read-only by multiple node.
ReadWriteMany (RWX) - mounted as read-write by multiple node.

• PV's can be

1) Dynamically Provisioned using StorageClasses 2) Static Provisioned by an administrator

• K8s creates PV automatically

PVC created

↓
Storage
classes

↓
PV create
automatically

• Admin creates PV manually

Admin

↓
Create PV

↓
PVC uses PV

• PV with YAML File

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: "/mnt/data"
```

• PVC with YAML File

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

- To Create PVC `kubectl apply -f PersistentVolumeClaim.yaml`
- To View PVC `kubectl get pvc`
- To Describe PVC `kubectl describe pvc my-pvc`
- To Delete PVC `kubectl delete pvc my-pvc`

Pod YAML with PV and PVC

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx:latest
      volumeMounts:
        - name: my-persistent-storage
          mountPath: "/usr/share/nginx/html"
  volumes:
    - name: my-persistent-storage
      persistentVolumeClaim:
        claimName: my-pvc
```

* Storage Lifecycle

- 1) Create PV (PV created)
- 2) Create PVC (PVC requests Storage)
- 3) Binding (PVC ↔ PV)
- 4) Pod uses PVC (Pod → PVC → PV)

Service

- It provides stable network endpoint to access pods.
- It exposes pods and provides a permanent IP address and DNS name, even if Pods are recreated.

Why we need Service?

- Pods are ephemeral

eg. Pod-1 IP = 10.244.1.5 if pod crashes Pod-1 deleted

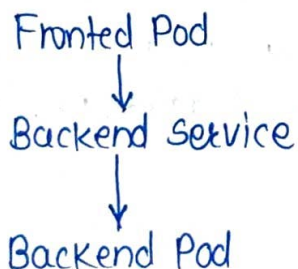
New pod :- Pod-2 IP = 10.244.1.8

The IP changes

Applications can't rely on Pod IPs because they are temporary

A Service provides Stable IP + Stable DNS

- Solution With Service



The Frontend talks to service, not directly to Pods.

Types of Services :-

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80 (service port)
      targetPort: 80 (container port)
  type: ClusterIP (service type)
```

1) ClusterIP (default)

Exposes the app inside the cluster. external user cant access it.
eg. Backend APIs, Databases, Internal microservices.

2) NodePart

Exposes the app on port of every node.
eg. Testing, Lab env, small clusters.

3) Load Balancer

Creates an external load balancer through the cloud provider.
eg. product' app, Public APIs, web app.

4) ExternalName

Maps a k8s service to an external DNS name
connecting external database, third-party APIs

- ★ To Create service `kubectl apply -f service.yml`
- To view service `kubectl get svc`
- To Describe service `kubectl describe svc nginx-service`
- To Delete service `kubectl delete svc nginx-service`
- To view endpoints `kubectl get endpoints` (Shows which pods are behind the service)

Ingress

- It manages external HTTP/HTTPS access to services inside a cluster.
- It acts like a smart router that directs incoming web traffic the correct K8s service based on hostname or URL path.

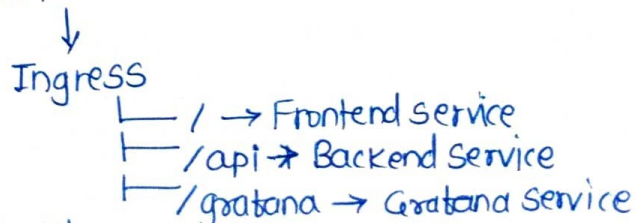
Why we need Ingress?

Imagine we have 3 app. Frontend App, Backend API, Monitoring Dashboard.

without ingress: LoadBalancer service + Frontend
 _____ " _____ + Backend
 _____ " _____ + Grafana

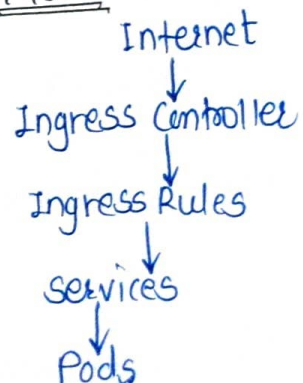
Problems: multiple L.B, Higher cloud cost, Difficult management

Solution :- Ingress Internet



One entry point can route traffic to multiple services.

Flow

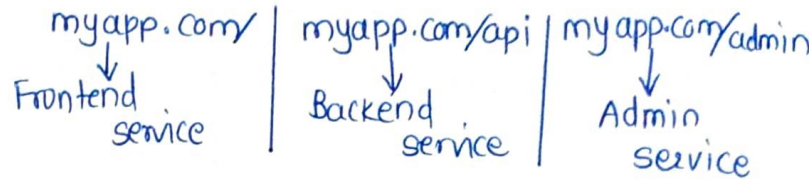


YAML

```
apiVersion: networking.k8s.io/v1
Kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
    - host: myapp.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

Path-Based Routing

Different URLs route to different services.

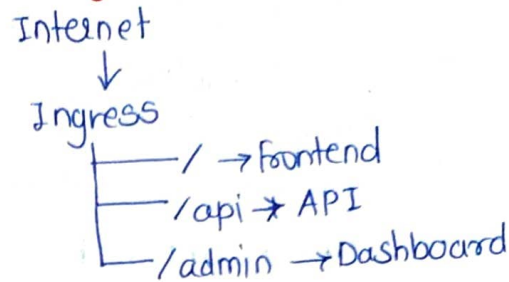


Real-world Scenario

Your company wants to hosts:

- React Frontend
- Node.js API
- Gatsby dashboard

Use Ingress.



- To create Ingress `kubectl apply -f ingress.yml`
- To view Ingress `kubectl get ing`
- To describe Ingress `kubectl describe ingress app-ingress`
- To Delete Ingress `kubectl delete ingress app-ingress`

StatefulSet

- It is used to manage the stateful applications that require stable pod names, persistent storage, ordered deployment and scaling.
- Deployments work very well for Stateless app.

- Stateful Pods: eg. `mysql-0` `mysql-1` `mysql-2` Names remain consistent.

ordered Deletion:

Deletion happens in reverse order
`mysql-2` → `mysql-1`
↓
`mysql-0`

- Stable network Identity: eg. `mysql-0.mysql-service` ✓ each pod gets predictable DNS name
1. ——— ✓ Useful for databases, distributed system
2. ——— cluster communication.
- Persistent Storage: eg. `mysql-0` → `PV-0` ✓ each pod gets its own persistent volume
`mysql-1` → `PV-1` ✓ if Pod restarts
`mysql-2` → `PV-2` `mysql-0` → Same PV attached
✓ Data remains available
- Ordered deployment: eg. `mysql-0`
↓
`mysql-1`
↓
`mysql-2`
✓ Pods are created sequentially

Statefulset Architecture

Statefulset

Headless Service

mysql-0

PV-0

mysql-1

PV-1

mysql-2

PV-2

Statefulset require Headless Service

YAML

✓ It provides direct DNS access to individual pods.

ClusterIP: None

- To Create statefulset `kubectl apply -f statefulset.yaml`
- To view ————— `kubectl get sts`
- To Describe statefulset `kubectl describe sts mysql`
- To Delete statefulset `kubectl delete sts mysql`

Deployment



Stateless app

(dj, Flask, Springboot)

Statefulset



Stateful app

(mysql, mongoDB)

Configmap

- It is used to store non-sensitive configuration data as key-value pairs
- It allows you to separate application configuration from container images making app. to manage and update.
- Configuration is hardcoded inside the pod definition.

Problems: Need to edit YAML every time configuration changes.

Same image cant to reused easily across environments.

Difficult to manage at scale.

Solution:

Application



ConfigMap



Configuration Values

Now Configuration is stored separately from the application.

- Real-use case

Suppose an application needs :

Database Host = mysql

Environment = Production

Log Level = INFO

[Instead of embedding these values in container image, store them in ConfigMap]

ConfigMap Architecture

ConfigMap

1 — DB_HOST=mysql

1 — APP_ENV=production

1 — LOG_LEVEL=info

To Create Configmap `kubectl apply -f config.yml`

`kubectl get cm` (To view configmap)

`kubectl describe configmap app-config` (To describe configmap)

`kubectl delete configmap app-config` (To delete configmap)

Secret ConfigMap YAML | Pod with ConfigMap

apiVersion: v1 Kind: ConfigMap metadata: name: app-config data: DB_HOST: mysql APP_ENV: production LOG_LEVEL: info	apiVersion: v1 Kind: Pod metadata: name: app-pod spec: containers: - name: app image: nginx env: - name: DB_HOST valueFrom: configMapKeyRef: name: app-config key: DB_HOST
---	---

Secret

- It is used to store sensitive information such as:
 - ✓ Passwords
 - ✓ API keys
 - ✓ Database credentials
 - ✓ SSH keys
 - ✓ TLS certificates.

and makes it available to pods without hardcoding it in application code or YAML file.

YAML

```
apiVersion: v1
Kind: Secret
metadata:
  name: mysql-secret
type: Opaque (default Secret type used for arbitrary Key-value pairs)
data:
  Username: YWRtaW4=
  password: cGZzc3dvmjM=
```

Pod YAML with SECRET

- Base 64 Encoding
secrets are stored as Base64 encoded values

* `echo -n "admin" | base 64`

→ `YWRtaW4=`

* `echo -n "pass123" | base 64`

→ `cGZzc3dvmjM=`

To view secrets

- `kubectl get secrets`
To Describe secrets
- `kubectl describe secret mysql-secret`
To Delete secrets
- `kubectl delete secret mysql-secret`

```
env:
- name: DB_USER      -name: DB_PASSWORD
  valueFrom:         valueFrom:
    secretKeyRef:     secretKeyRef:
      name: mysql-secret
      key: Username   name: mysql-secret
                      key: Password.
```

Probes

Probes are health checks used by K8s to determine status of container.
It ensures pod or container is healthy, ready to receive traffic or needs to be restarted.

Why we need Probes?

Imagine an applicatⁿ Pod is running.

But internally, Application Hung, Database Connectⁿ lost, memory leak.

K8s sees the container as running but doesn't know the app. is Unhealthy

Probe solve this problem.

Types of Probes:-

1) Liveness Probe:- ✓ check whether the app. is alive.

YAML

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

✓ It determines whether a container should be restarted.

Wait before starting probe checks i.e wait 10 sec then start checking

How often the probe runs. i.e check every 5 sec

2) Readiness Probe:- ✓ checks whether the app. is ready to receive traffic

YAML

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

3) Startup Probe:- checks whether a slow-starting app. has started successfully.
They protect slow-starting app from premature restarts.

* Why needed?

Some application take a long time to start: Java applicatⁿ, Spring boot app, Large database

YAML

No. of failures before actⁿ
Fail 30 times
Then take actⁿ

```
StartupProbe:
  httpGet:
    path: /health
    port: 8080
  FailureThreshold: 30
  periodSeconds: 10
```

• Without startup probe	• With Startup Probe
Applicat ⁿ starting... Liveness probe fails Container Restarted Infinite restart loop	wait for startup ↓ Applicat ⁿ Ready ↓ Enable Liveness Probe

Taints and Toleration.

Taints are applied to Nodes to repel Pods, it restricts pod being scheduled on specific nodes.

Toleration are applied to pods to allow them to be scheduled on a tainted Nodes.

Why we need?

Suppose we have:

node → 1 General App.

node → 2 Database Server.

You don't want normal app. pods to run on database node.

Apply a taint on Node-2. ⇒ Node-2

↓
No Pods allowed.

Only Pods with a matching toleration can run there.

• How Taints work.

• `kubectl taint nodes worker-node1 app=db:NoSchedule.`
node-name key value Effect

Node-1
↓
Only DB Pods allowed.

• `kubectl taint nodes worker1 env=production:NoSchedule.`

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  tolerations:
    - key: "app"
      operator: "Equal"
      value: "db"
      effect: "NoSchedule"
  containers:
    - name: mysql
      image: mysql:8
```

pod YAML ✱ To remove Taint

`kubectl taint nodes worker1 app=db:NoSchedule-`

This removes taint

✓ Now pod can run on

app=db:NoSchedule

✓ tainted Nodes.

Horizontal Pod Autoscaler (HPA)

It automatically increases or decreases the no. of pods in a Deployment, StatefulSet or ReplicaSet based on resource usage as CPU or memory utilization.

Why we need HPA?

Imagine your application normally receives:

100 users → 3 pods are enough.

Suddenly traffic increases → 10,000 users.

Now 3 pods can't handle the load

Without HPA → High CPU

↓
Slow Response

↓
Application failure

With HPA: → CPU Increases

↓

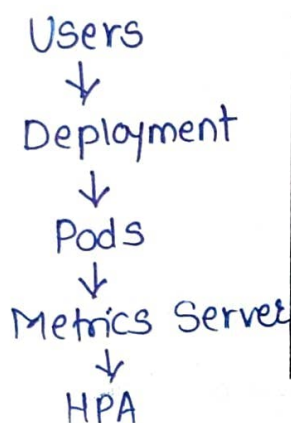
HPA detects load

↓

Created more pods

↓
App. remain available.

• HPA Architecture



✓ HPA continuously checks metrics and adjust replicas.

✓ HPA requires a metrics provider. most used → Metrics server

• kubectl get top nodes

• kubectl top pods

} if these works, metrics server is running
To check metrics.

YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: utilization
        averageUtilization: 50
```

* Metrics Used by HPA

1) CPU Utilization :-

averageUtilization: 50

2) Memory Utilization :-

resource:

name: memory

3) Custom metrics :-

eg. Req. per sec, queue length, App-specific metrics
Often integrated with: Prometheus & KEDA

• To create HPA `kubectl apply -f hpa.yaml`

To view HPA `kubectl get hpa`

To Describe HPA `kubectl describe hpa nginx-hpa`

To Delete HPA `kubectl delete hpa/nginx-hpa`

Vertical Pod Autoscaler (VPA)

- It automatically adjusts the CPU and memory requests/limits of pod based on actual resource usage.
- It automatically increase or decrease the CPU & memory assigned to Pod instead of creating pods.

Why we need VPA?

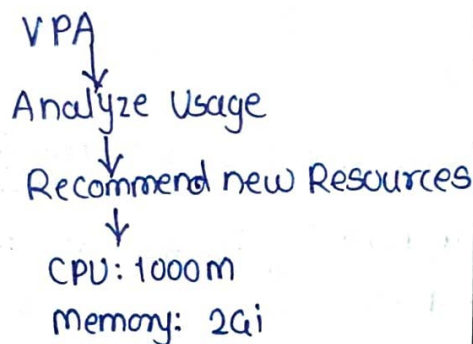
Suppose an applicatⁿ is deployed with → CPU: 200m
memory: 256Mi
but actual usage is → CPU = 1000m
memory = 2Gi

Results → High CPU Usage, OOMKilled Errors, Poor Performance.

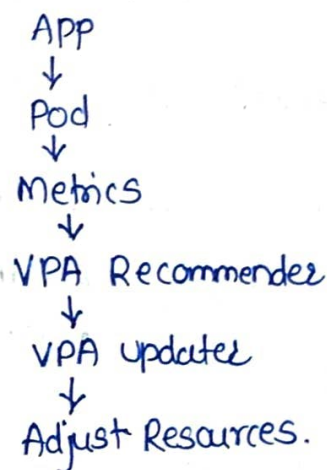
VPA Analyzes Usage and recommends or automatically update resources.

Here app. needs more resources.

With VPA



★ VPA Architecture



YAML

```
apiVersion: autoscaling.k8s.io/v1
Kind: VerticalPodAutoscaler
metadata:
  name: nginx-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  updatePolicy:
    updateMode: Auto
```

★ Updates Modes

- 1) **Off**
updateMode: off
✓ only recommendations.
✓ No automatic updates
- 2) **Initial**
updateMode: Initial
✓ Recommendation applied only during pod creation
✓ Existing pods remain unchanged.
- 3) **Auto**
updateMode: Auto
✓ VPA automatically update resources.
✓ Pod may be restarted.

★ To create VPA `kubectl apply -f vpa.yaml`

To view VPA `kubectl get vpa`

To Describe VPA `kubectl describe vpa nginx-vpa`

To Delete VPA `kubectl delete vpa nginx-vpa`

VPA Use Cases :-

Databases :- eg. mysql, PostgreSQL

They are difficult to scale horizontally

VPA helps by increasing CPU & memory.

RBAC (Role-Based Access Control)

- It allows administrators to define permissions for users, groups or service accounts and restrict access to Kubernetes resources.
- It is K8S authorization mechanism used to control who can do what in a cluster.

Why we need RBAC?

Imagine cluster with multiple teams: Dev Team, QA Team, Devops Team.

✓ Without RBAC:- everyone can - Delete pod, Access secrets, Delete Deployments

This is security risk.

✓ With RBAC

Developer	QA	Devops
1 View Pods	1 View Logs	1 Full Access

✓ Each user gets only the permission they need.

★ RBAC Components :-

1) Role - It defines permission with a specific namespace.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: Pod-reader
rules:
- apiGroups: [ "" ]
  resources: [ "Pods" ]
  verbs: [ "get", "list", "watch" ]
```

→ User can get pods, list pods, watch pods
User cant delete pods or create pods
only in dev namespace

- ✓ Namespace Scope
- ✓ Limited Access
- ✓ Used in one Namespace

2) ClusterRole - It defines permission across the entire cluster.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: Pod-reader
rules:
- apiGroups: [ "" ]
  resources: [ "Pods" ]
  verbs: [ "get", "list", "watch" ]
```

ClusterRole is not limited to a single namespace.

- ✓ cluster Scope
- ✓ Global Access
- ✓ Used Across Cluster

3) RoleBinding - It assigns a Role to user, Group or ServiceAccount.

```
apiVersion: rbac.authorization.k8s.io/v1
Kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
Subjects:
- kind: User
  name: John
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Flow:- John
↓
RoleBinding
↓
Role
↓
Read Pods.

- ✓ Namespace level
- ✓ Binds Role
- ✓ Limited scope

4) ClusterRoleBinding - Assigns a ClusterRole across the entire cluster.

```
apiVersion: rbac.authorization.k8s.io/v1
Kind: ClusterRoleBinding
metadata:
  name: admin-binding
Subjects:
- kind: User
  name: admin-user
roleRef:
  kind: ClusterRole
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
```

→ Entire cluster

- ✓ Cluster Level
- ✓ Binds clusterRole
- ✓ Full scope

* Common Verbs:- Permission are defined using verbs.

Verbs:

- | | |
|----------|---------------------|
| - get | - Read one resource |
| - list | - view resource |
| - watch | - monitor changes |
| - create | - Create resources |
| - update | - modify resources |
| - delete | - Remove resources |
| - patch | - partial update |

Verbs:

- "*" -

→ Full Access

Service Account + RBAC.

✓ RBAC is often used with Service Accounts.



✓ This controls what the pod can access.

* Commands.

Kubectl auth can-i list pods (can I list pods)

Kubectl auth can-i delete deployments --as=john (check another user)

Kubectl get roles (view roles)

Kubectl get clusterroles (view clusterroles)

Kubectl get rolebindings (view rolebinding)

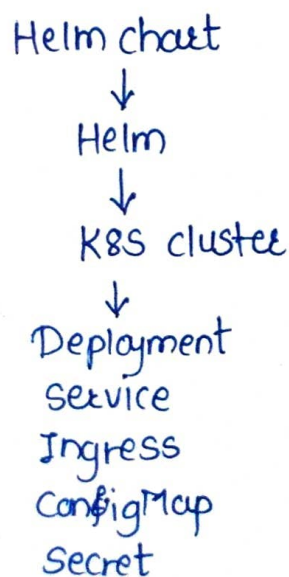
Kubectl get clusterrolebindings (view clusterrolebinding)

Helm

✓ Package manager for K8s.

✓ It helps to you install, upgrade, rollback, and manage K8s applications using reusable templates called charts.

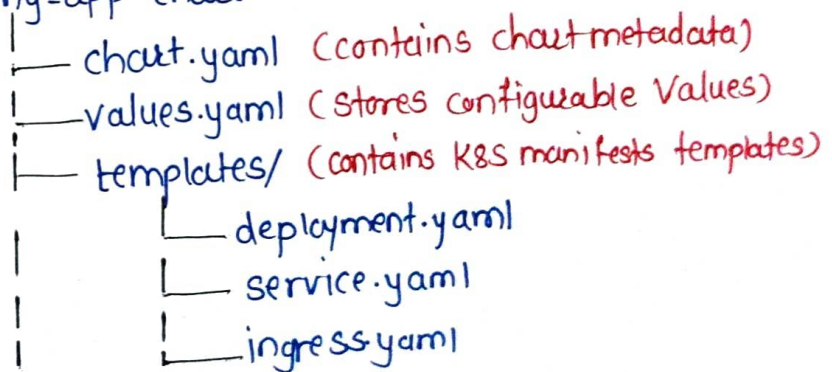
* Helm Architecture



1. Chart.

✓ It is package containing K8s manifests & templates.

eg. my-app-chart



✓ It is collection of K8s resource together.

2. Release

It is running instance of Helm chart.

eg. `helm install my-nginx bitnami/nginx`
 Release chart

3. Repository

- It stores chart.

eg. Bitnami charts, Artifact Hub

- `helm repo add bitnami https://charts.bitnami.com/bitnami` (Add repo)

- `helm repo update` (update)

- * `helm create nginx-helm` (To create chart)

- `helm install package nginx-helm nginx-helm` (To install chart)

- `helm uninstall nginx-helm` (To uninstall chart)

- `helm upgrade nginx-helm ./nginx-helm` (To update)

- `helm rollback nginx-helm 1` (Return to revision 1)

- `helm list` (to list releases)